

Entregable fase 4: Informe de validación y optimización de tienda online

Shomara Rivera

10 de julio de 2026

Índice

1. Resultados de consultas con datos de prueba	2
2. Explicación de índices creados y su impacto	3
3. Pruebas de procedimientos almacenados (escenarios de error)	4
3.1. Intento de reseña sin compra previa	4
3.2. Intento de compra sin stock suficiente	4
3.3. Intento de agregar producto duplicado	4
4. Propuestas de mejoras adicionales	5

Lista de códigos

1. Consulta de productos disponibles	2
2. Consulta de pedidos pendientes	2
3. Consulta de promedio de calificaciones	3
4. Error por falta de compra	4
5. Error por falta de stock	4
6. Error por producto duplicado	4

1. Resultados de consultas con datos de prueba

Al realizar las tres consultas del entregable anterior usando los datos de prueba que añadí, pudimos ver que el sistema responde de forma correcta y nos devuelve las tablas que esperamos; esto nos sirve para asegurar que la estructura soporta las operaciones de la tienda.

1. Listar productos disponibles por categoría, ordenados por precio: Esta consulta combina los productos con sus respectivas categorías usando un JOIN, mostrando de menor a mayor los artículos que sí tienen inventario.

Código 1: Consulta de productos disponibles

```
1 SELECT p.nombre AS producto, c.nombre AS categoria, p.precio,
   p.stock
2 FROM productos p
3 JOIN categorias c ON p.id_categoria = c.id_categoria
4 WHERE p.stock > 0
5 ORDER BY p.precio ASC;
```

producto	categoria	precio	stock
Cable USB-C	Accesorios	200.00	100
Poco X3	Teléfonos	5000.00	40
iPhone 13	Teléfonos	15000.00	20
MacBook Air	Laptops	20000.00	10

2. Mostrar clientes con pedidos pendientes y total de compras: Aquí agrupamos los pedidos con estado pendiente para identificar qué usuarios tienen compras en proceso, el conteo se hace de forma individual gracias al GROUP BY.

Código 2: Consulta de pedidos pendientes

```
1 SELECT c.nombre AS cliente, c.correo, COUNT(pe.id_pedido) AS
   total_pedidos_pendientes
2 FROM clientes c
3 JOIN pedidos pe ON c.id_cliente = pe.id_cliente
4 WHERE pe.estado = 'pendiente'
5 GROUP BY c.id_cliente;
```

cliente	correo	total_pedidos_pendientes
---------	--------	--------------------------

Pablo Hernandez	pablo@email.com		1	
Jhovanny Pérez	jhovanny@email.com		1	
+-----+-----+-----+-----+				

- Reporte de los 5 productos con mejor calificación promedio en reseñas: Esta consulta calcula el promedio de estrellas usando la función `AVG`; el filtro `LIMIT 5` recorta la tabla para mostrarnos solo el top de los mejores.

Código 3: Consulta de promedio de calificaciones

```

1 SELECT p.nombre AS producto, AVG(r.calificacion) AS
   promedio_calificacion
2 FROM productos p
3 JOIN resenas r ON p.id_producto = r.id_producto
4 GROUP BY p.id_producto
5 ORDER BY promedio_calificacion DESC
6 LIMIT 5;
7

```

+-----+-----+	
producto	promedio_calificacion
+-----+-----+	
iPhone 13	5.000000
Dell XPS 13	5.000000
+-----+-----+	

2. Explicación de índices creados y su impacto

Como vimos en las diapositivas de clase, para mejorar el rendimiento de la base de datos agregamos índices en las columnas que más vamos a usar para buscar datos, como un índice funciona como el índice de un libro, permite que MySQL vaya directo a la fila en lugar de leer toda la tabla.

- **idx_producto_nombre:** Creado sobre el nombre de los productos, ayuda a que el buscador de la tienda sea más rápido.
- **idx_producto_categoria:** Creado sobre la llave foránea de categorías para filtrar rápido.
- **idx_pedido_cliente:** Creado en los pedidos para cargar el historial del cliente sin demoras.

Para comprobar que sí funcionan, utilizamos el comando `EXPLAIN` antes de hacer un `SELECT`, pues este comando le pregunta a MySQL cómo planea buscar el dato.

id	select_type	table	type	possible_keys	key	key_len
1	SIMPLE	productos	ref	idx_producto_nombre	idx_producto_nombre	452

Al ver la columna llamada **key**, podemos ver que MySQL sí utilizó nuestro índice; y en la columna **rows**, dice que solo tuvo que leer 1 fila para encontrar el resultado, demostrando que la consulta está optimizada.

3. Pruebas de procedimientos almacenados (escenarios de error)

Para asegurar que se cumplan las restricciones del profesor, ejecutamos los procedimientos almacenados usando el comando **CALL**, intentando meter datos falsos para ver si el sistema nos detenía.

3.1. Intento de reseña sin compra previa

Intentamos que el cliente 2 hiciera una reseña del producto 1, el cual nunca ha comprado.

Código 4: Error por falta de compra

```
1 CALL sp_registrar_resena(5, 'Excelente celular', 1, 2);
```

El procedimiento detectó que no había registros de compra y bloqueó la inserción, arrojando el mensaje: *Error: Solo puedes hacer reseñas de productos que hayas comprado.*

3.2. Intento de compra sin stock suficiente

Intentamos registrar un pedido por 1000 unidades de un producto que solo tiene 20 en stock.

Código 5: Error por falta de stock

```
1 CALL sp_registrar_pedido(1, 1, 1000, 15000.00);
```

El procedimiento usó un IF para revisar el stock y, al ver que no alcanzaba, abortó la venta arrojando: *Error: No hay suficiente stock para este producto.*

3.3. Intento de agregar producto duplicado

Intentamos registrar un iPhone 13 cuando ya existe otro igual en la misma categoría.

Código 6: Error por producto duplicado

```
1 CALL sp_agregar_producto('iPhone 13', '...', 15000, 10, 1);
```

El sistema contó los nombres repetidos y lanzó la alerta: *Error: Ya existe un producto con ese nombre en esta categoría.*

4. Propuestas de mejoras adicionales

Al ir haciendo este proyecto me di cuenta de un par de cosas que podríamos implementar a futuro basándonos en los temas que vimos en clase:

- **Uso de vistas (VIEWS):** Durante la Fase 3 me di cuenta de que hacer consultas con JOIN puede ser un poco complejo, como mejora, propongo guardar las consultas más difíciles (como el reporte de clientes con pedidos pendientes) dentro de una VIEW, así, cualquier usuario podría ver la información rápidamente sin tener que escribir todo el código SQL cada vez.
- **Uso de triggers (Disparadores):** Para tener aún más seguridad, propongo agregar un Trigger antes de insertar reseñas, aunque el procedimiento ya revisa que el cliente haya comprado el artículo, un Trigger nos ayudaría a asegurar que nadie meta un número negativo o un 10 en la calificación de estrellas, garantizando que siempre sea del 1 al 5.